Università degli Studi di Milano-Bicocca
Corso di Laurea Magistrale in Data Science
Data Management

Formula 1 2026: A Multi-Source Data Management Pipeline

Acquisition, Integration and Analysis of Relational and Semi-Structured Racing Data

Author
Filippo Avon — Matricola 886229

September 2026

Table of Contents

1. Introduction **3**

1.1 Project Components and Roles 3
1.2 Research Questions 3

2. Datasets and Acquisition **3**

2.1 Source 1 — Kaggle: jtrotman/formula-1-race-data 3
2.2 Source 2 — TracingInsights (github.com/TracingInsights/2026) 3
2.3 Acquisition Design Notes 4

3. Process and Pipeline **4**

3.1 Storage 4
3.2 Data Cleaning 4
3.3 Integration and Enrichment 4
3.4 Data Quality Assessment 5

4. Software Architecture **6**

4.1 MySQL Relational Schema 6
4.2 MongoDB Document Structure 7

5. Results: The Four Research Questions **8**

RQ1 — Fastest Pit Stop of the 2026 Season 8
RQ2 — Highest Peak Speed of the 2026 Season 9
RQ3 — Best Race Pace on Medium Tyres, Barcelona-Catalunya GP 10
RQ4 — Best Qualifying-to-Race Comeback 11

6. Conclusions **13**

6.1 Summary 13
6.2 Limitations 13
6.3 Future Work 13

7. References **14**

1. Introduction

1.1 Project Components and Roles

This project was developed individually by Filippo Avon as part of the Data Management course (Master's degree in Data Science, Università degli Studi di Milano-Bicocca). The project implements a complete data management pipeline — acquisition, storage, profiling, cleaning, integration/enrichment, and quality assessment — applied to the 2026 Formula 1 season, combining a relational dataset with a semi-structured (JSON) dataset obtained from two independent, publicly available sources.

The pipeline was implemented in Python, orchestrated from a Jupyter notebook, with two dedicated database management systems (one relational, one document-oriented) run as reproducible Docker containers.

1.2 Research Questions

The project addresses four research questions, deliberately chosen to exercise both data sources — individually and jointly — and to require different query paradigms (SQL and MongoDB aggregation):

- RQ1 — Which driver recorded the fastest pit stop of the 2026 season, and in what circumstances (time, lap, race, circuit, team)?
- RQ2 — What is the highest instantaneous speed recorded by any car during the 2026 season?
- RQ3 — Which driver had the best race pace on Medium-compound tyres at the Spanish/Catalunya Grand Prix (Barcelona)?
- RQ4 — Which driver achieved the best qualifying-to-race position "comeback", both as a single-race record and as a season-long average?

RQ1 and RQ4 are answered with SQL queries against the relational (MySQL) database, since all the required information such as pit stop durations, qualifying and race positions were available there. RQ2 and RQ3 require data that exists only in the semi-structured (MongoDB) source, raw telemetry and tyre compound per lap, respectively , are answered with MongoDB aggregation/Python processing.

2. Datasets and Acquisition

2.1 Source 1 — Kaggle: jtrotman/formula-1-race-data

A relational dataset distributed as a set of CSV files (races, results, qualifying, lap_times, pit_stops, driver_standings, constructor_standings, constructor_results, sprint_results, drivers, constructors, circuits, status), covering every Formula 1 season from 1950 to the current 2026 season. It was downloaded programmatically using the official kagglehub Python library, authenticated via a personal Kaggle API token. Because the dataset spans the entire history of the championship, it was filtered to the 2026 season only during the Data Cleaning phase (see Section 3.2).

2.2 Source 2 — TracingInsights (github.com/TracingInsights/2026)

A GitHub repository publishing session-level Formula 1 data derived from the FastF1 library, organized as <Grand Prix>/<Session>/<file>.json, including per-lap telemetry sampled at approximately 3.7 Hz, lap times with tyre compound and speed-trap data, weather, race control messages, and circuit corner coordinates. The complete file listing was retrieved via the GitHub REST API — the Git Trees API in recursive mode, with a Contents API fallback used when the Trees API returned an error, which was observed to occur intermittently on a repository of this size. Every JSON file was then downloaded from the raw.githubusercontent.com CDN using a parallel thread pool. A total of 47,771 files were retrieved.

2.3 Acquisition Design Notes

Both acquisition scripts were built to respect the rate limits of their respective services (GitHub's unauthenticated 60 requests/hour limit and Kaggle's authentication requirements were both handled explicitly), to run unattended, and to be fully reproducible: rerunning either script downloads the same data with no manual steps.

3. Process and Pipeline

3.1 Storage

The two sources were kept in two different DBMS, matching the nature of their data:

- MySQL 8.0 (relational) for the Kaggle data — database f1_kaggle. Well suited to the strongly relational structure of the source (numeric surrogate keys, foreign-key relationships between races, results, drivers, constructors and circuits).
- MongoDB 7.0 (document-oriented) for the TracingInsights data — database f1_tracinginsights. Well suited to the heterogeneous, session-dependent structure of the source, where forcing every file into a fixed relational schema would have been artificial and lossy.

Both databases run as Docker containers defined in a single docker-compose.yml file, which also serves the project's reproducibility requirement: any evaluator can recreate an identical environment with a single command.

3.2 Data Cleaning

Two issues were identified during profiling and corrected before further processing:

- MySQL — season scope. The Kaggle dataset spans 1950–2026; it was filtered to the 2026 season only, using races.year to identify the relevant raceId values and propagating that filter to every race-level table (results, qualifying, lap_times, pit_stops, driver_standings, constructor_standings, constructor_results, sprint_results), while reference/lookup tables (drivers, constructors, circuits, status) were kept in full, since they act as join targets and carry no season concept themselves.
- MongoDB — collection fragmentation. The initial loading logic used each JSON filename as the target collection name. Because TracingInsights stores one telemetry file per lap (e.g. 1_tel.json, 2_tel.json, …), this fragmented telemetry into approximately 150 separate collections instead of one unified collection. The loading logic was rewritten to route each document by the depth of its file path instead of its filename, which correctly distinguishes session-level files (3 path levels: Grand Prix / Session / file) from driver-level, per-lap files (4 path levels: Grand Prix / Session / Driver/ file). This consolidated all telemetry into a single telemetry collection (with driver_code and lap added as explicit fields), corrected a missing driver_code field in laptimes, excluded one file pattern found to be inconsistent across sessions (cor.json, present in only 5 of 69 sessions versus 69/69 for the equivalent corners.json — most likely a stale or duplicate artifact from the source repository), and isolated the two dataset-metadata files (a machine-readable data dictionary and a JSON Schema for the telemetry files) into a dedicated _metadata collection, separate from the race data itself. This reduced MongoDB from 174 fragmented collections to 8 clean ones.

3.3 Integration and Enrichment

The two sources do not share a common key out of the box: MySQL identifies drivers and races with numeric surrogate keys (driverId, raceId), while MongoDB uses human-readable identifiers (three-letter driver codes, Grand Prix names). An automated mapping procedure was built to reconcile them, and its accuracy was explicitly measured — a required deliverable of the project.

Driver code mapping

MySQL drivers.code (restricted to drivers with at least one 2026 result) was compared against MongoDB's driver_code field. Result: 23 of 23 championship drivers (100%) matched directly, with no manual correction required. Nine additional codes present only in MongoDB (ARO, BEG, BRO, CRA, FOR, HER, HIR, IWA, VES) correspond to reserve/test drivers who appear in pre-season testing telemetry but never in an official race — they are absent from Kaggle's results table by construction of that source, not due to an integration failure.

Race name mapping

MySQL races.name was compared against MongoDB's grand_prix field. Result: 12 of 12 races that had already taken place at the time of acquisition matched (11 directly, 1 via a manual mapping rule — "Barcelona Grand Prix" in MongoDB corresponds to "Barcelona-Catalunya Grand Prix" in MySQL). Three pre-season testing events present only in MongoDB were explicitly excluded from the race-level join, as they have no counterpart in Kaggle's race calendar. Races scheduled later in the 2026 calendar and not yet run at acquisition time appear only in MySQL and were correctly left unmatched.

Enrichment

Once validated, the mapping was applied to write raceId and driverId directly into every relevant MongoDB document, using batched update_many operations grouped by distinct key value — a few dozen database calls in total — rather than one update per document, which would not have scaled to the 45,964-document telemetry collection.

3.4 Data Quality Assessment

Following the explicit requirement to assess data quality both before and after the integration/enrichment step, this assessment is split into two parts: the intrinsic quality of each source taken independently (immediately after Cleaning, before any cross-source key is written), and the outcome of integration itself.

Before Integration — Each Source Independently

MySQL (season-filtered, 9 tables): 0% missing cells across all tables. Referential integrity was checked explicitly for every foreign-key relationship in the schema (15 checks in total — e.g. results.raceId → races.raceId, lap_times.driverId → drivers.driverId): every single one resolved at 100%, with zero orphaned rows.

MongoDB (8 collections): completeness of the natural-key fields already present before enrichment (grand_prix, session, driver_code, lap, depending on the collection) was checked across 17 field/collection combinations: all at 100%. No duplicate documents were found on the natural key (grand_prix + session + driver_code + lap) in telemetry, and no duplicate rows were found in MySQL results (same driver and race appearing twice).

In short: both sources are, on their own, complete and internally consistent. This is an important baseline for interpreting the integration results below — it rules out pre-existing data quality problems as an explanation for any gap observed after integration.

After Integration / Enrichment

- Completeness of enrichment — raceId coverage in MongoDB ranged from 77.1% (telemetry) to 100% (session_laptimes); driverId coverage was 98.7–99.0% (laptimes, telemetry).
- Consistency / integration match rate — 12/12 races matched (100%, 3 testing sessions correctly excluded); 23/23 championship drivers matched (100%).

Because both sources were already shown to be complete and internally consistent on their own, every enrichment gap can be attributed specifically to differing coverage between the two sources, rather than to any pre-existing defect: missing raceId values correspond exactly to the three pre-season testing events (which have no MySQL race record), and missing driverId values correspond exactly to reserve/test drivers absent from Kaggle's results table.

4. Software Architecture

The pipeline is implemented as a sequence of standalone Python scripts, run interactively from a Jupyter notebook, following a clear separation of concerns: one script per pipeline stage, each idempotent and independently rerunnable. Figure 1 summarises the end-to-end architecture.

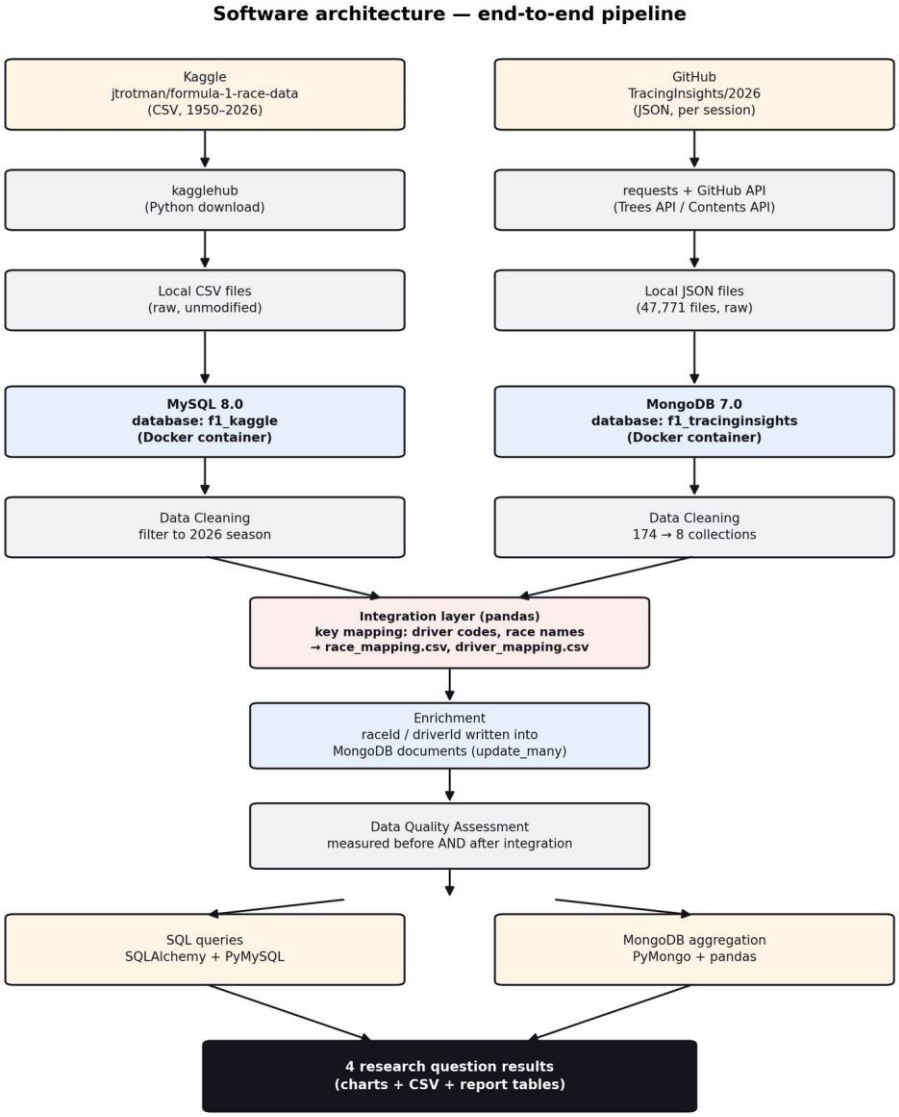


Figure 1 — End-to-end sokware architecture, from source APIs to research-question results.

4.1 MySQL Relational Schema

Figure 2 shows the simplified entity-relationship structure of the f1_kaggle database after the season filter was applied. All fact tables (results, qualifying, lap_times, pit_stops, the two standings tables, sprint_results) reference races.raceId; results and its dependents also reference drivers.driverId and constructors.constructorId. The four lookup tables (drivers, constructors, circuits, status) were intentionally kept unfiltered, since restricting them to 2026-only values would have broken joins with no benefit.

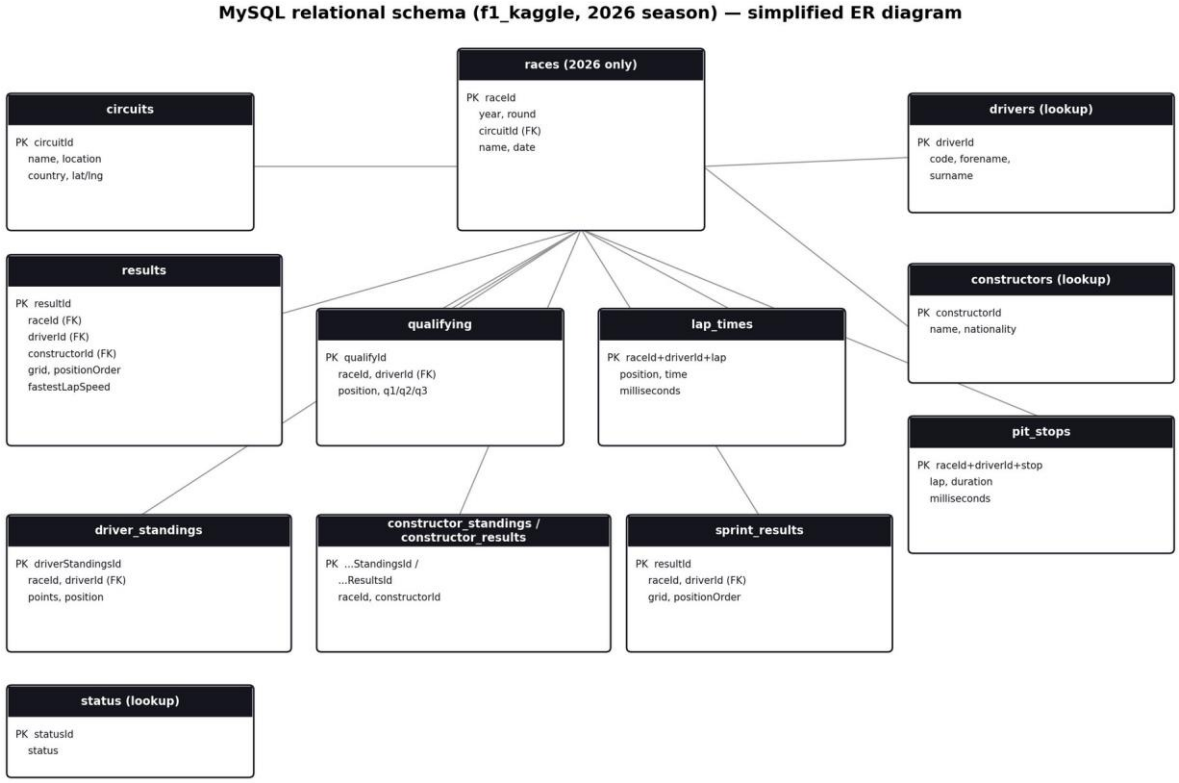


Figure 2 — Simplified ER diagram of the MySQL schema (f1_kaggle, 2026 season).

4.2 MongoDB Document Structure

Figure 3 shows the eight collections of the f1_tracinginsights database after cleaning, with their approximate document counts and the fields added during enrichment (marked with an asterisk). Note the columnar (array-of-values) structure of laptimes and weather, inherited directly from the source format: each document represents one driver/session (or one session, for weather), and the arrays are indexed by lap number.

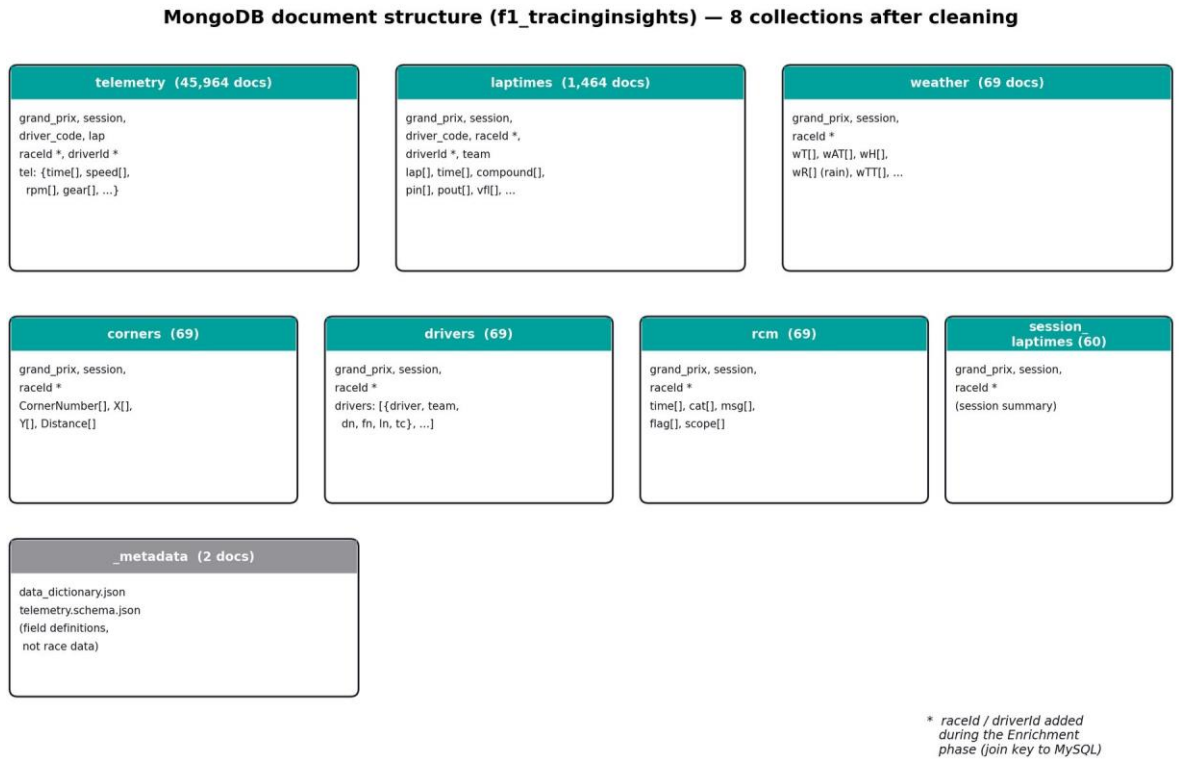


Figure 3 — MongoDB document structure (f1_tracinginsights), 8 collections aker cleaning.

5. Results: The Four Research Questions

RQ1 — Fastest Pit Stop of the 2026 Season

Query type: SQL (MySQL), joining pit_stops, drivers, results, races and circuits, ordered by pit stop duration in milliseconds.

The fastest pit stop of the 2026 season was performed by Franco Colapinto (Alpine F1 Team), in 12.092 seconds, at lap 8 of the Dutch Grand Prix, on the Circuit Park Zandvoort (Zandvoort, Netherlands). The second-fastest stop of the season was also recorded at the same event, by Arvid Lindblad (RB F1 Team), in 12.338 seconds — notably, both of the season's two fastest stops occurred at the same race, roughly 4.7 seconds ahead of the third-placed result.

#	Driver	Team	Race (lap)	Duration
1	F. Colapinto	Alpine F1 Team	Dutch GP (L8)	12.092 s
2	A. Lindblad	RB F1 Team	Dutch GP (L5)	12.338 s
3	N. Hülkenberg	Audi	Dutch GP (L19)	17.082 s
4	V. Bottas	Cadillac	Miami GP (L30)	17.173 s
5	L. Stroll	Aston Martin	Dutch GP (L35)	17.200 s

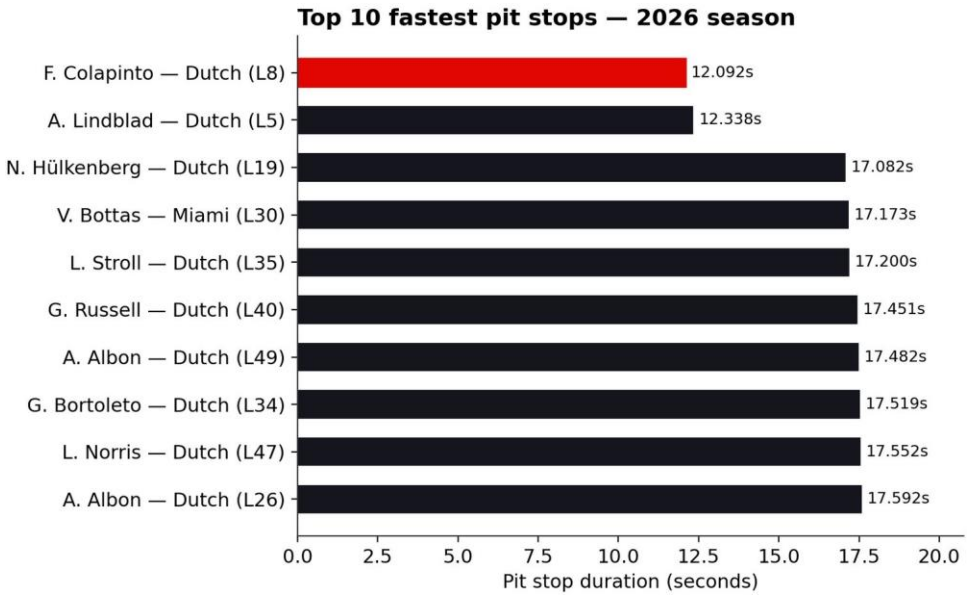


Figure 4 — Top 10 fastest pit stops of the 2026 season.

Note: this duration is the full pit-lane time — from pit-lane entry to exit — as recorded in the source data, not the shorter stationary time alone (car jacked up, tyres changed) that real Formula 1 typically reports and which is usually around 2–3 seconds. The two are not directly comparable; see Section 6.2 for further discussion.

RQ2 — Highest Peak Speed of the 2026 Season

Query type: MongoDB aggregation, using the native $max array operator on the per-lap telemetry field tel.speed (raw FastF1 data sampled at ~3.7 Hz), which is a considerably more precise source for this question than the four discrete speed-trap fields available in laptimes.

The highest instantaneous speed recorded during the 2026 season was 360.0 km/h, by Gabriel Bortoleto (Audi), at lap 2 of the Barcelona-Catalunya Grand Prix. Four of the season's ten highest readings were recorded in that same session, within its first five laps.

#	Driver	Team	Grand Prix (lap)	Peak speed
1	G. Bortoleto	Audi	Barcelona (L2)	360 km/h
2	G. Bortoleto	Audi	Barcelona (L3)	359 km/h
3	E. Ocon	Haas F1 Team	Barcelona (L2)	358 km/h
4	N. Hülkenberg	Audi	Canadian (L22)	357 km/h
5	C. Sainz	Williams	Hungarian (L68)	357 km/h

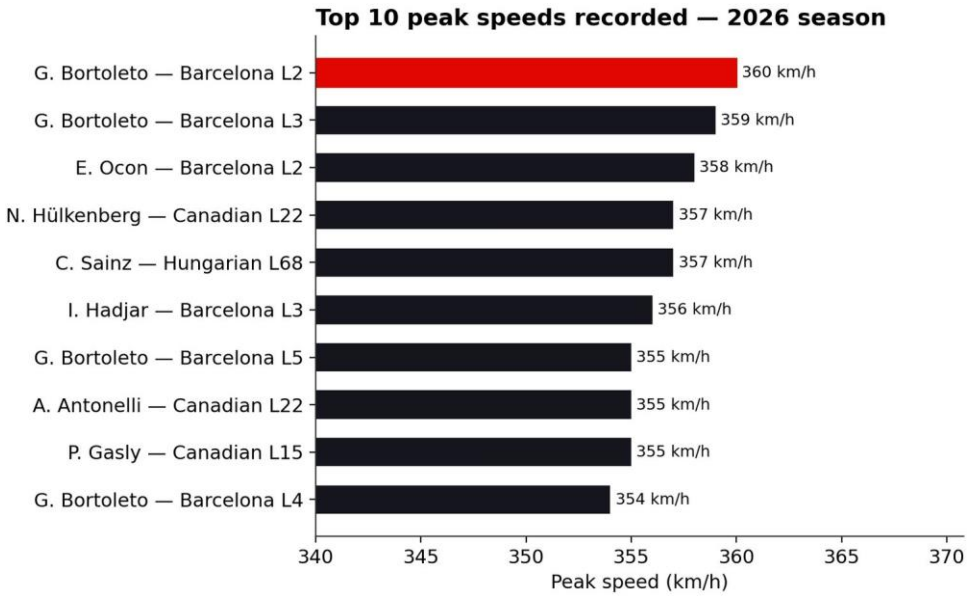


Figure 5 — Top 10 peak speeds recorded during the 2026 season.

Note: the 3 best top speeds so far in 2026, and even more, are recorded in the first opening laps of the Grand Prix, which usually has a lot of fights, traffic and cars are with a full tank of fuel. Even though this is relatively unexpected, this is not an error.

RQ3 — Best Race Pace on Medium Tyres, Barcelona-Catalunya GP

Query type: MongoDB (laptimes collection), filtering each driver's lap array to compound == "MEDIUM", excluding pit in/out laps, laps deleted by stewards, and laps flagged as timing-inaccurate by FastF1, then averaging the remaining lap times.

The best average race pace on Medium-compound tyres at the Barcelona-Catalunya Grand Prix was set by Lewis Hamilton (Ferrari), averaging 81.117 seconds per lap over 12 qualifying laps (best single lap: 80.633 s). Max Verstappen (Red Bull Racing) was second, averaging 82.005 s over a much larger sample of 37 clean laps.

#	Driver	Team	Clean laps	Avg. lap time	Best lap
1	L. Hamilton (HAM)	Ferrari	12	81.117 s	80.633 s
2	M. Verstappen (VER)	Red Bull Racing	37	82.005 s	80.230 s
3	C. Sainz (SAI)	Williams	4	82.262 s	82.061 s
4	G. Russell (RUS)	Mercedes	10	82.629 s	81.479 s
5	K. Antonelli (ANT)	Mercedes	11	82.981 s	82.042 s

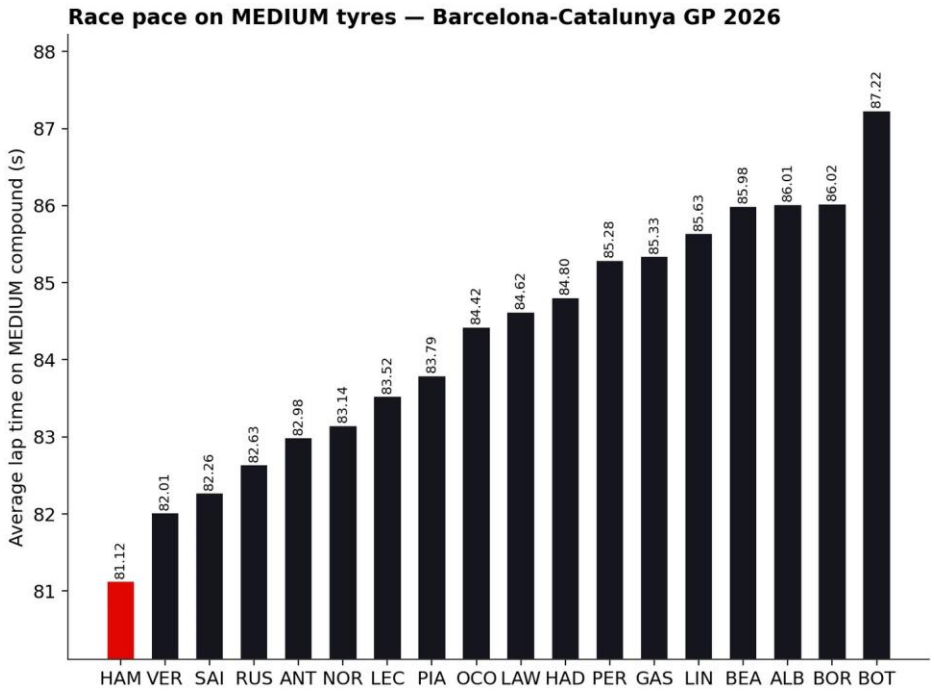


Figure 6 — Average race pace on MEDIUM tyres, Barcelona-Catalunya GP 2026, all 18 drivers with at least one clean lap.

Note on sample size: Hamilton's headline result rests on only 12 clean laps, versus 37 for Verstappen — a direct consequence of differing tyre strategies during the race. This is reported transparently (the clean-lap count is included in every row) rather than hidden behind the average alone, precisely so a reader can judge how much weight to give the comparison.

RQ4 — Best Qualifying-to-Race Comeback

Query type: SQL (MySQL), joining qualifying and results on race and driver, computing positions_gained = qualifying_position − race_position. Two complementary readings are reported: the single largest comeback of the season, and the driver with the best average comeback across all races.

Single-race record

The largest comeback of the 2026 season was achieved by Fernando Alonso, who started the Monaco Grand Prix (7 June 2026) in 21st position on the grid and finished 10th — a gain of 11 positions in a single race.

#	Driver	Race	Grid	Finish	Gained
1	F. Alonso	Monaco GP	21	10	+11
2	F. Colapinto	British GP	19	9	+10
3	F. Alonso	Dutch GP	18	9	+9

Season-long average

Considering the entire 2026 season, Fernando Alonso also leads on average consistency, gaining 3.25 positions per race across 12 races — narrowly ahead of Sergio Pérez (3.08 positions/race over 12 races).

#	Driver	Races	Avg. positions gained
1	F. Alonso	12	+3.25
2	S. Pérez	12	+3.08
3	L. Stroll	11	+2.45
4	F. Colapinto	12	+1.92
5	E. Ocon	12	+1.50

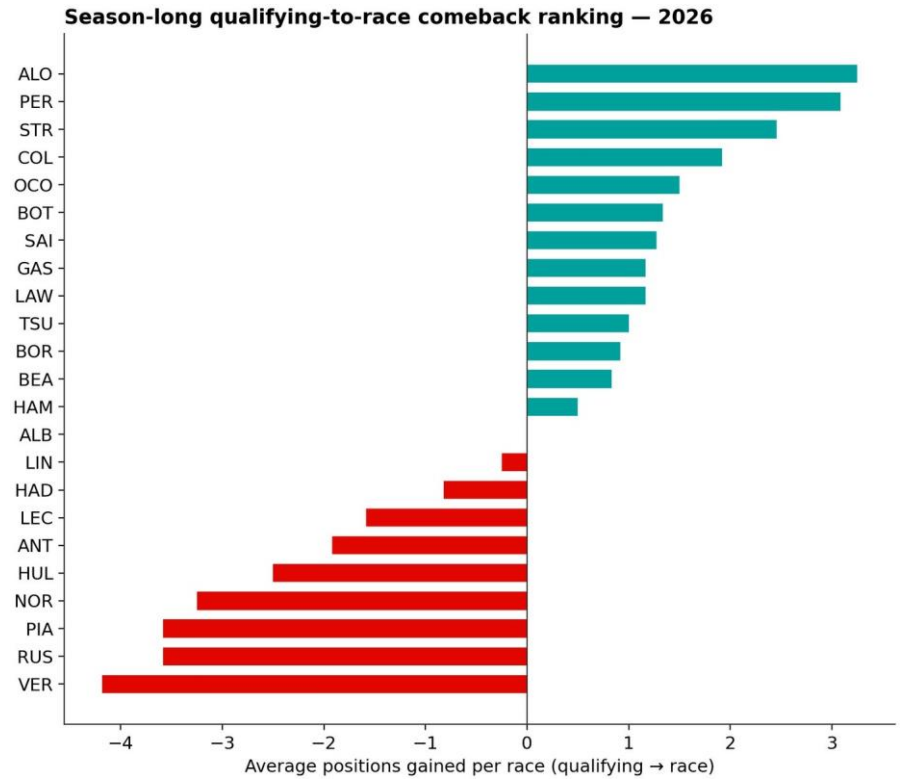


Figure 7 — Season-long average qualifying-to-race position change, all drivers, 2026.

Alonso's result on both readings is a genuine finding, not an artefact of one metric: the same driver tops both the single-race record and the season-long average, which strengthens confidence that it reflects a real pattern of race-day performance relative to qualifying pace rather than a one-off result.

6. Conclusions

6.1 Summary

This project built a complete, reproducible data management pipeline combining a relational dataset (Kaggle, 1950–2026 F1 results, filtered to the 2026 season) and a semi-structured dataset (TracingInsights, session-level JSON derived from FastF1 telemetry), stored respectively in MySQL and MongoDB running as Docker containers. Integration between the two sources — which do not share a common identifier system out of the box — was automated and its accuracy explicitly measured: 100% of championship drivers and 100% of races that had taken place were matched, with every remaining gap traced to a documented, structural limitation of one source rather than to an unexplained error. Four research questions were answered, two requiring pure SQL and two requiring the richer, telemetry-level detail available only in the document store, demonstrating the practical value of combining both DBMS rather than relying on either source alone.

6.2 Limitations

- Pit stop duration scope (RQ1): the duration recorded in the Kaggle pit_stops table measures the full pit-lane time — from the moment the car crosses the pit-lane entry line to the moment it crosses the exit line — not the shorter stationary time the car spends being serviced in the box alone, which in real Formula 1 is typically around 2–3 seconds. This is why the season's "fastest" result (12.092 s) is far longer than the sub-2-second stationary-time records widely cited in the sport; the two measure genuinely different things, and are not directly comparable.
- Uneven sample sizes (RQ3): the tyre-compound pace comparison mixes drivers with very different numbers of clean qualifying laps (from 4 to 37), a consequence of differing race strategies rather than a data quality defect, but one that affects how much statistical confidence should be placed in any single driver's average.
- Season still in progress: at the time of acquisition, several 2026 races (e.g. Abu Dhabi, Las Vegas, Singapore) had not yet been run, and were consequently present only in the MySQL race calendar and absent from MongoDB. This is an expected, time-bound gap rather than an integration failure, but it does mean the season-long comeback ranking (RQ4) reflects a partial season, not the final championship standings.
- Manual mapping rule: one race-name discrepancy ("Barcelona Grand Prix" vs. "Barcelona-Catalunya Grand Prix") required a manually curated mapping rule rather than a fully automatic match. While this was the only case found in the current data, the same class of discrepancy could recur for future races and would need the same manual review step.

6.3 Future Work

- Re-run the pipeline once the 2026 season is complete, to obtain a full-season comeback ranking and a complete peak-speed comparison across every circuit.
- Extend the spatial dimension of the integration: TracingInsights' corners collection carries circuit corner coordinates (X, Y, distance from start/finish) that could be combined with MySQL's circuits.lat/lng using a spatial extension (e.g. MySQL's native spatial data types or PostGIS) to analyse braking points and cornering speed relative to circuit geometry — an area the project brief explicitly encouraged but which was out of scope for the current four research questions.

- Extend the driver/race key-mapping procedure into a small reusable lookup table maintained across seasons, rather than a per-season CSV, to reduce the manual-mapping burden identified in Section 6.2.

7. References

- Kaggle dataset: jtrotman, "Formula 1 Race Data", https://www.kaggle.com/datasets/jtrotman/formula-1-race-data
- TracingInsights, "2026" (GitHub repository), https://github.com/TracingInsights/2026
- FastF1 Python library (upstream data source for TracingInsights), https://docs.fastf1.dev/
- GitHub REST API documentation, https://docs.github.com/rest
- kagglehub Python library, https://github.com/Kaggle/kagglehub
- Docker Compose documentation, https://docs.docker.com/compose/
- MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/refman/8.0/en/
- MongoDB 7.0 Manual, https://www.mongodb.com/docs/v7.0/
- pandas documentation, https://pandas.pydata.org/docs/
- SQLAlchemy documentation, https://docs.sqlalchemy.org/
- PyMongo documentation, https://pymongo.readthedocs.io/